Ballarch: Boundaries in Time and Honest Outcomes, from Register-Transfer Logic to Services

Vladislav Tomilov
Independent researcher

Ballarch repository: `https://github.com/4wl2d/Pokeball`

October 2026
Specification version 2.0.0-rc.1

Abstract

Software architecture styles draw boundaries in space: they constrain which code may depend on which. They say little about *when* an effect may leave a component, or what it may claim after a timeout, when retrying can charge a customer twice and reporting failure can hide a charge that happened.

We present Ballarch, which adds two *boundaries in time*: the *commit horizon*, the atomic commit of the decision that requests an effect, and the *send horizon*, the first moment the request may have left the runtime. Transactions, transports and durable execution use these instants locally; Ballarch makes them boundaries of every interaction. Each piece of state has one owner that decides in pure functions. A runtime commits state and requested effects atomically, and ends every outbound request in exactly one outcome, Done, NotDone or Unknown, chosen from its phase and its port's effect class. For operations made of several requests, a knowledge algebra over Kleene's strong three-valued logic bounds what a component may claim. It is sound, and exact for independent calls; with a durable call ledger, a composite that replies with settled knowledge is itself an honest component.

The rules are stated once. Level-specific mechanisms check the call-protocol rules and the algebra in a Kotlin runtime, C firmware and SystemVerilog blocks. Composition across levels is not yet tested. In a checkout benchmark without idempotency keys, naive versions in four styles (a timeout means failure) violated a requirement in 12–94% of runs. Careful versions and Ballarch violated none while the delivery horizon held, but did beyond it; with keys and an attempt timeout, Ballarch did not in sampled runs, but careful versions that reconcile by status after a crash did. Ballarch's runtime takes over this care, except the reconciliation, at a cost in code, framework types, total complexity and change size.

Keywords: software architecture; fault tolerance; idempotency; three-valued logic; output commit; hardware/software co-design.

1 Introduction

A payment provider that does not deduplicate requests receives a charge request and charges the card; the answer is then lost. The client times out. What should it record?

- If it retries, the customer may be charged twice.
- If it reports a failure, a charged customer is told that nothing happened.

The only honest answer is a third one: the charge *may* have happened, and someone has to find out. The "third outcome" of remote procedure calls is old knowledge [11]. Distributed-systems testing

records such operations as indeterminate [45, 46], and transaction standards report heuristic and hazard outcomes [87]. HTTP advises a client not to retry a non-idempotent request automatically unless it knows that the request is in fact idempotent or that it was never applied; HTTP/2 lets a server supply evidence of the latter, and gRPC retries transparently on such evidence [25, 28, 83]. Durable-execution engines record when each activity is scheduled and when it is dispatched [80]. Hardware solved a related problem long ago: processors retire instructions in order so that exceptions are precise [76], and memory types that model devices forbid speculative or repeated accesses [4].

Architecture styles for application software do not, however, place these facts in the architecture:

- Layered and Clean Architecture [56], Ports and Adapters [20] and domain-driven design [27] constrain dependencies.
- Actors [6, 17, 39] and the Elm architecture [22] give state a single owner.
- Sagas [30] and the transactional outbox [73] keep state and messages consistent.

The outbox fixes when a message may leave: after the state that requests it commits. None of them says what a component may claim once a request may have left it, for example after a timeout. Each feature author decides. In our benchmark (Section 8), naive versions in four styles, which share one policy (a timeout or I/O error means that the payment failed), charged customers twice or reported charged orders as failed in 12–94% of runs without idempotency keys. Careful versions of the same four styles, written with each style's own tools, were as correct as Ballarch while the provider's declared delivery horizon held, but each kept that correctness in its own feature code.

This paper presents Ballarch, an architecture whose rules make that knowledge structural. Ballarch keeps established mechanisms (single owners, pure decision functions, commit then dispatch, adapters at the edge) and cites them as such. Its contributions are the following:

1. **Boundaries in time** (Section 3). Every interaction crosses two instants in order: the *commit horizon* and the *send horizon*. The instants are known from transaction processing, outboxes, transport protocols and durable execution (Section 9). We make them normative boundaries of a general architecture style, alongside the spatial ones: every implementation, whether a service, firmware or a circuit, must expose them as observable events, and the outcome rules are derived from them.

2. **Outcomes from phase and effect class** (Section 4). A runtime, not each feature, transport or adapter, chooses each request's single outcome from its durable phase and its port's effect class. It never reports NotDone without evidence and never re-sends a non-idempotent request. Transports and durable-execution engines already tell a request that never left from one that may have run [2, 25, 28, 82, 83]. Ballarch derives the outcome from an effect class declared for each port, refuses retries of NonIdempotent ports by construction rather than by configuration, and gives the decider a typed three-valued completion that composes.

3. **A knowledge algebra with composition closure** (Section 5). Knowledge of a composite operation is a pair (*all*, *any*) over Kleene's strong logic, with absorbing compensations; it refines the heuristic-outcome merge of transaction managers. Because effects and compensations are encoded as read-once formulas, known properties of Kleene's logic make sound parts compose into a sound whole whose two coordinates are each exact when the calls are independent. A composite that counts what it may still do as unknown and replies only with settled knowledge is itself an honest constituent of a larger operation, given honest adapters, absorbing compensations, and a call ledger kept for as long as the calls' effects persist.

4. **One rule set, checked by level-specific mechanisms** (Section 6). Fourteen rules are stated once, without reference to language, transport or storage. References for services (Kotlin),

firmware (C) and synchronous hardware (SystemVerilog) check the call-protocol rules and the algebra by mechanisms suited to their level; the Kotlin reference also checks the other rules that have a mechanism. Each level is checked on its own: no reference composes levels.

5. **A benchmark of where the care lives** (Section 8). One checkout feature is written in four styles, each in a naive and a careful version, and in Ballarch. It runs against providers without and with idempotency keys, under three fault profiles, with one crash per run and an exhaustive single-crash sweep, and is complemented by seven seeded bugs, four change requests and a measurement of runtime overhead. The benchmark separates whether a design is correct from where it keeps the knowledge that makes it correct, and reports what Ballarch costs next to what it changes.

We evaluate the architecture with an Alloy model of the call protocol, exhaustive checks of the algebra, fault exploration at each level, including a composite of composites at the service level, and the benchmark. All artifacts are public [84].

2 Model

Balls and deciders. A *Ball* is a single-writer owner of state, addressed by a type and a key. Its type is defined by a *decider*, a set of pure functions:

- `decide(state, request, ctx)` returns *Accept*(state′, reply plan, outputs) or *Reject*(reason);
- `complete(state, completion, ctx)` and `observe(state, notice, ctx)` return a step (state′, outputs).

Outputs are *calls* to ports, *replies*, *notices* and *cancellations*. The context supplies time, identity and fresh call keys, so deciders read nothing ambient. The idea of a decider is not new [18, 22]; what is new is what the runtime does with its outputs.

Ports and effect classes. A *port* is a typed request/response contract owned by its destination. Its *effect class* states what repeating a request does there:

- Safe: no effect;
- Idempotent: the destination deduplicates by the call's identity;
- NonIdempotent: every delivery may take effect.

The names follow HTTP's safe and idempotent methods [28]. HTTP defines idempotence by the intended effect of repeated identical requests; Idempotent here requires that the destination deduplicate by the call's identity, as an idempotency key or token does [50, 77]. RPC interface languages already declare such classes per operation, to choose HTTP methods (protobuf [70]) or to tell clients which operations are safe to retry (Smithy [77]), and the Model Context Protocol lets a server mark each LLM tool with read-only, destructive and idempotent hints [59]; Ballarch also derives each call's outcome from its class (Section 4). In hardware the same distinction appears as Normal versus Device memory [4], and as prefetchable PCIe memory ranges, whose reads have no side effects [68].

Obligations and outcomes. A committed call is an *obligation*: the runtime must deliver exactly one completion for it to the caller's `complete`, by a finite deadline. The completion's outcome is one of the following:

- Done(value): the destination processed the request; the value may describe a business refusal, such as a declined charge;
- NotDone(reason): there is evidence that the request had no effect;

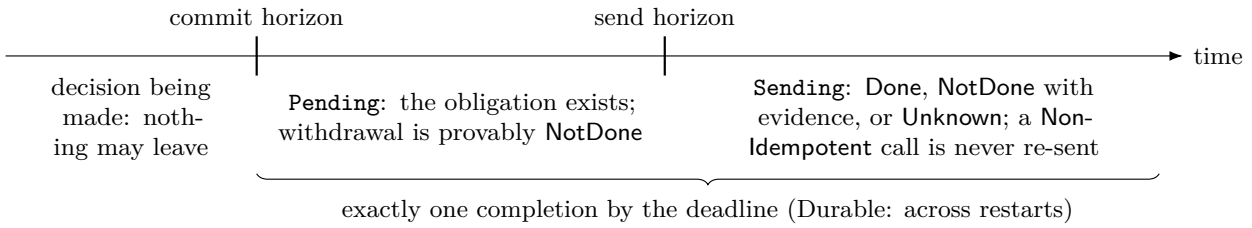


Figure 1: The two boundaries in time that every call crosses. Before the commit horizon there is nothing to know; between the horizons non-occurrence can still be proved; after the send horizon only evidence about every attempt, or later reconciliation, establishes what happened to an effectful call.

- Unknown(reason): the request may or may not have taken effect.

Profiles. Under the *Transient* profile nothing survives a crash. Under the *Durable* profile the committed state survives, together with the call ledger, which records each call's phase and the effect class it was committed with, recently answered requests and unacknowledged notices.

3 Boundaries in Time

Spatial boundaries constrain which code may refer to which. Ballarch adds two boundaries that constrain *when* effects and knowledge may cross (Figure 1):

Commit horizon. The atomic commit of the decision that requests a call. Before it, the call is part of a decision that can still vanish, and nothing may leave (commit before dispatch). After it, the call is an obligation.

Send horizon. The first moment the request may have left the runtime. Before it, the call can be withdrawn truthfully. After it, NotDone requires evidence covering every attempt (a refusal, or an adapter's evidence of non-application), except for a Safe call, which can have had no effect.

An implementation must make each horizon an event that its checks can observe:

- in the Kotlin runtime: a store write, then a write-ahead `Sending` record written before the transport is invoked;
- in firmware: the CRC-checked NVM record, then the write-ahead record before the UART transmission;
- in a synchronous circuit: the clock edge, then the first cycle in which the request can no longer be withdrawn from the bus (the `valid && ready` handshake, or the assertion of VALID on AXI).

A service runtime in any language can report both horizons in a common conformance trace, as `Committed` and `CallSent` events (Section 6).

The mechanisms behind both horizons are established:

- the output-commit rule of rollback recovery [26, 78];
- write-ahead logging [60];
- operating systems that hold external output until it no longer depends on uncommitted state [63, 64];
- in-order retirement in processors [76].

Situation	Safe	Idempotent	NonIdempotent
Destination answered	Done	Done	Done
Deadline or cancel before the send horizon	NotDone	NotDone	NotDone
Evidence of non-application, one attempt	NotDone	NotDone	NotDone
Same, after more than one attempt	NotDone	retry or Unknown	–
No answer, attempts and time left	retry	retry	Unknown
No answer, no attempts or time left	NotDone	Unknown	Unknown
Durable restart after the send horizon	re-send or NotDone	re-send or Unknown	Unknown
Durable restart before the send horizon	send	send	send

Table 1: The runtime's classification of a call's single outcome. A Safe call that gets no answer ends NotDone(NoAnswer), because it can have had no effect. Evidence that one attempt was not applied says nothing about earlier attempts, so after several attempts of an Idempotent call it does not justify NotDone; a NonIdempotent call has only one attempt. A report that arrives after the call resolved is dropped.

Real-time architectures specify component interfaces in the temporal domain [48], but for timing predictability rather than for knowledge about effects. The instants themselves are known in practice. Transaction processing defers real actions, which cannot be undone, until commit [33], and the transactional outbox commits messages together with state [73]. Service-design guidance commits each local atomic phase before a foreign state mutation, and counts the request as committed once the first such mutation is made [50]. Durable-execution engines record when an activity is scheduled and when it is dispatched to a worker [80], and AWS Lambda durable functions checkpoint an at-most-once step before it runs [2]. What we add is to make both instants normative boundaries of a general architecture style, which every implementation must expose as observable events, and to derive from them outcome rules that keep Unknown apart from failure (Section 4). The service-design guidance above, for example, marks a timed-out non-idempotent call as failed. Section 9 compares the outcome rules with the retry rules of transports and durable execution.

4 Outcomes from Phase and Effect Class

Table 1 gives the classification that a conforming runtime applies. Two rules carry its weight:

R7 (honest outcomes). NotDone is produced only with evidence: the call was never sent, the destination refused it, an adapter reported evidence covering every attempt that may have been delivered, or the port is Safe and so has no effect.

R8 (no blind re-send). A NonIdempotent call is sent at most once, including after a restart. Before each send, the runtime durably records that the call may have been sent.

Retries belong to the runtime and are allowed only for Safe and Idempotent ports. Adapters report each attempt once and never retry. A conforming runtime classifies a call by the effect class recorded when it was committed, never by its port's binding after a later deploy, so rebinding a port as Idempotent cannot turn a possibly sent NonIdempotent call into a re-send. The Kotlin runtime does so; the firmware reference records no class with its calls (Section 6). Table 2 lists all fourteen rules with the mechanisms that check them.

The decider sees one completion per call and must handle all three outcomes. The type system makes Unknown a case, not an exception that each call site maps differently:

```
when (val outcome = completion.outcomeOf(Mailer)!!) {
    is Outcome.Done -> step(ReminderState.Sent)
    is Outcome.NotDone -> step(ReminderState.NotSent(outcome.reason.toString()))
    is Outcome.Unknown -> step(ReminderState.MaybeSent) // never re-send blindly
}
```

5 Knowledge of Composite Operations

A Ball often performs one operation through several calls: a flight and a hotel, a reservation and a charge. Workflows usually report such operations as succeeded or failed. After a partial or uncertain effect, both labels can be false.

Definition 1 (Truth values)**.** Let $\mathbb{T} = \{1, 0, \bot\}$ with Kleene's strong connectives [47] and the information order $\bot \sqsubseteq 1$, $\bot \sqsubseteq 0$. A value t *admits* a Boolean b if $t = \bot$ or t is the truth value of b (1 for true, 0 for false). A pair of values admits a pair of Booleans if each coordinate admits the corresponding Boolean.

Definition 2 (Constituents and knowledge)**.** The *constituents* of an operation are the calls it needs, requested or not. A claim about the operation is about its *final* effects, those that hold once none of its requests can still take effect. At any moment, the *effect* $f_i \in \mathbb{T}$ of a constituent is:

- 1 if it completed Done with an answer that, by the port's contract, means that the requested effect happened;
- 0 if it completed NotDone, if it completed Done with an answer meaning that the effect did not happen (a recorded refusal), or if the Ball can no longer request it for this operation;
- $\bot$ if it completed Unknown, if it completed Done with an answer that leaves the effect open, if it was requested and its completion has not arrived, whether or not it was sent, or if the Ball has not requested it but still may.

If the Ball may request the same effect again after a NotDone, its attempts, including those it may still make, form one constituent, whose effect f_i is the disjunction of theirs, each defined as above. A compensation is *absorbing* if its effect means that no effect of the original request exists or can still take effect. For example, the destination fences the request's reference, as the anti-suspension fence of Try-Confirm/Cancel implementations does [3], or the original request's *delivery horizon*, the age after which the destination no longer applies it, has passed. In Pardon and Pautasso's design, a participant cancels a reservation that is not confirmed within its timeout and refuses a later confirmation [66]; that timeout is the confirmation's delivery horizon. Let $c_{i1}, \ldots, c_{im}$ be the effects of a constituent's compensation attempts, each defined like f_i, and let u_i be their disjunction, joined with $\bot$ while the Ball may still compensate the constituent. If the compensations are absorbing, the constituent's *net effect* is $n_i = f_i \wedge \neg u_i$. The *knowledge* of an operation with constituents $1..k$ is

$$K = (all, any) = \Big(\bigwedge_i n_i,\ \bigvee_i n_i \Big), \qquad K_\emptyset = (1, 0).$$

Counting every call that the Ball may still make as $\bot$ makes K a claim about final effects at every moment, not only at the end. Three-valued runtime verification treats the unobserved future in the same way: it gives a finite prefix a definite verdict only when every continuation agrees, so that no verdict is ever revised [8, 23]. Here the principle is applied to the effects of calls and compensations and computed truth-functionally, which is sound and, under the conditions of Proposition 2, exact. A report of the completions received so far is a different claim, and must

Rule	Binds	Statement (abridged)	Checked in this work by	Levels
R1 Single owner	author	one owner per fact; immutable values	bytecode rules; C purity lint; Yosys single-driver check	K C R
R2 Pure decider	author	no I/O, clocks, randomness, globals	bytecode rule; C purity lint; Yosys: no latch, flip-flops the only storage	K C R
R3 Serial processing	runtime	no concurrent or nested steps per instance	runtime guard; trace oracle T3	K
R4 Atomic commit	runtime	state and outputs in one atomic write	store design; Alloy P6; crash and power-loss exploration; Yosys: one rising clock edge, no input to an asynchronous pin	K C R
R5 Commit before dispatch	runtime	nothing leaves before its commit	Alloy P1; oracle T5; power-loss exploration; RTL benches; Yosys path check	K C R
R6 One completion per call	runtime	exactly one by the deadline	Alloy P2, L1; oracle T6; C exploration; RTL benches with phase deadlines	K C R
R7 Honest outcomes	runtime, adapter	NotDone only with evidence	Alloy P3, P5; oracle T7 with ground truth; C exploration; RTL bench	K C R
R8 No blind re-send	runtime, adapter	NonIdempotent sent at most once; write-ahead	Alloy P4; oracle T8; seeded bugs; C exploration; RTL bench	K C R
R9 One reply per request	runtime, author	at most one reply; overdue replies reported	preflight; oracle T9	K
R10 Explicit composition	author	interaction only via calls, replies, notices	kernel API; module-boundary and public-API checks	K
R11 Bounds and preflight	runtime	refuse before accepting; reject faulty decisions	preflight tests; C runtime tests	K C
R12 Read-only queries	author	reads do not change state	types; review	K
R13 Honest profile claims	runtime	no claim beyond the profile	review	–
R14 Honest composition	author	claims about composites follow the algebra	`Knowledge` type; trip and itinerary exploration; C algebra check and transfer exploration; RTL composite bench	K C R

Table 2: The rules of Ballarch Core 2.0 and the mechanisms that check them. Levels: the references that check the rule mechanically (K Kotlin service, C firmware, R RTL). T3–T9 are properties of the conformance trace (Section 6); the Kotlin oracles check the same properties. Seeded bugs test the checks at every level.

not be offered as the operation's outcome. An operation is *settled* when none of its calls is open and none can still be made; from then on, only Unknown completions and open answers leave a constituent at $\bot$.

Composition $(a_1, s_1) + (a_2, s_2) = (a_1 \wedge a_2,\ s_1 \vee s_2)$ is a commutative monoid with identity $K_\emptyset$, the knowledge of an operation that requested no effect. A constituent contributes the summand (n_i, n_i). A call to a composite port contributes a pair instead: the composite's settled knowledge if the call completed Done with it; $(0, 0)$ if the Ball has not made it and can no longer make it, or if it completed NotDone and was the only request that could start the operation; and $(\bot, \bot)$ otherwise: if it completed Unknown, was answered before the composite settled, is still open or may still be made, or completed NotDone while another request may have started the operation. Two replies about the same operation are never added. Seven values can occur:

- Done $(1, 1)$ and NotDone $(0, 0)$;
- Partial $(0, 1)$: some effects happened and some did not;
- NotAllDone $(0, \bot)$ and SomeDone $(\bot, 1)$;
- Unknown $(\bot, \bot)$;
- NoEffects $(1, 0)$.

Seen as one call, a composite reports $\pi(K)$: Done if $K = (1, 1)$, NotDone if $any = 0$ (this includes NoEffects), and Unknown otherwise.

Both propositions below rest on standard facts. Kleene's strong connectives are monotone in $\sqsubseteq$ [47]. On formulas in which every variable occurs once, they agree with supervaluation, in van Fraassen's sense [86]: the value is definite exactly when all ways of resolving the unknowns agree. This agreement is folklore; the proof of Proposition 2 includes it. Hardware engineers know the second fact from ternary (0, 1, X) simulation, which goes back to hazard detection in switching circuits [24]. Read $\bot$ as Verilog's `x`: the proof of Proposition 2 also shows why X-propagation is exact through logic without reconvergent fanout, and why it can be pessimistic with it. What is ours is the encoding of effects and absorbing compensations as such formulas, and the closure property that follows from it.

Proposition 1 (Soundness)**.** *Let b_i be the true final net effect of constituent i. If each n_i admits b_i, then* all *admits $\bigwedge_i b_i$ and* any *admits $\bigvee_i b_i$. The same holds if some summands are pairs (a_i, s_i), each admitting the conjunction and the disjunction of the true net effects of its own constituents.*

Proof. Kleene's strong connectives are monotone in $\sqsubseteq$ and agree with the Boolean ones on definite values. Replacing each $\bot$ among the n_i by the corresponding b_i refines every n_i to b_i, so $\bigwedge_i n_i \sqsubseteq \bigwedge_i b_i$, which is the claim for *all*; the claim for *any* follows in the same way. For pairs, each coordinate of $+$ is monotone, and by associativity the conjunction of the inner conjunctions is the conjunction over all inner constituents; the same holds for disjunctions. □

Proposition 2 (Coordinate-wise exactness)**.** *Let the effect of each call, including each repeated attempt and each compensation attempt, be the most informative value that admits every possibility for that call alone. Assume that the calls are* independent*: the combinations of true effects consistent with the outcomes are exactly the product of the per-call sets of possibilities. Then each coordinate of K is the most informative value that admits every such combination.*

Proof. For a formula φ over the calls' effects, let $S(\varphi) \subseteq \{0, 1\}$ be the set of its Boolean values over the consistent combinations. The most informative value admitting all of them, the supervaluation, is 1, 0 or $\bot$ as $S(\varphi)$ is $\{1\}$, $\{0\}$ or $\{0, 1\}$. We show by induction that, on a formula in which each variable occurs at most once, Kleene evaluation equals supervaluation. For a variable, this is the hypothesis. A call that the Ball may still make enters as a variable; since the Ball's later calls often depend on

earlier outcomes, independence then usually holds only once the operation is settled. For a constant it is immediate, which covers $K_\emptyset$. For $\neg\varphi$, $S(\neg\varphi) = \{\neg x : x \in S(\varphi)\}$. For $\varphi_1 \circ \varphi_2$ with $\circ \in \{\wedge, \vee\}$, the two sides share no variable, so by independence $S(\varphi_1 \circ \varphi_2) = \{x \circ y : x \in S(\varphi_1),\ y \in S(\varphi_2)\}$, and the nine cases are Kleene's tables. *all* and *any*, with each $n_i = f_i \wedge \neg(c_{i1} \vee \cdots \vee c_{im})$ substituted, and each f_i of repeated attempts written as the disjunction of their effects, are such formulas, because every attempt is a distinct call. Both hypotheses are needed. With a repeated variable, $x \vee \neg x$ evaluates to $\bot$ although its supervaluation is 1; in a circuit, this is reconvergent fanout. With correlated calls, the product step fails: if a Ball books a hotel only after its flight ends NotDone, then while the flight is open *all* evaluates to $\bot$, although no consistent combination has both effects, so its supervaluation is 0. □

Corollary 1 (Composition closure)**.** *Assume that:*

(i) adapters and destinations report honestly (R7, and Done answers follow the port's contract);
(ii) every compensation counted in a net effect is absorbing;
(iii) the Ball keeps its state and call ledger for as long as the effects of its calls persist: under the Durable profile, or under the Transient profile only if a crash also clears those effects;
(iv) the Ball derives every claim about a composite operation from K (rule R14), and answers the request that started the operation only once the operation is settled (a deferred reply*), with K itself or with $\pi(K)$.*

Then:

(a) at every moment, K admits the conjunction and the disjunction of the constituents' true final net effects;
(b) the reply admits them as well;
(c) the summand that the call to the composite contributes to its caller's knowledge, as described after Definition 2, admits them too.

By induction on the depth of nesting, every claim derived this way, at every depth, admits the truth.

Proof. (a) By (iii), every call made for the operation whose effect can still exist is recorded in the Ball's state, so it is counted in some f_i or u_i. By (i), the value of each such call admits its true effect at every moment: honest reporting covers Done and NotDone, a call that can no longer be made never takes effect, and every other case is $\bot$. By monotonicity, a disjunction of repeated attempts admits whether one of them takes effect, so each f_i admits whether its constituent's request takes effect. While the Ball may still compensate, the disjunction with $\bot$ keeps u_i at $\bot$ unless a compensation already took effect, so u_i admits whether some compensation of constituent i takes effect. Because the compensations are absorbing (ii), the true net effect is $b_i = \beta_i \wedge \neg\gamma_i$, where β_i says whether the constituent's request takes effect and γ_i whether one of its compensations does. By monotonicity, n_i admits b_i, and Proposition 1 gives (a). (b) By (iv), a reply with K is (a) at the moment of settlement. For $\pi(K)$, let the composite's true effect as one call be any Boolean B with $\bigwedge_i b_i \le B \le \bigvee_i b_i$ when $k \ge 1$, such as 'every effect happened' or 'some effect happened', and $B = 0$ when $k = 0$. If $\pi(K) =$ Done, then $all = 1$, so $\bigwedge_i b_i = 1$ and $B = 1$. If $\pi(K) =$ NotDone, then $any = 0$, so $\bigvee_i b_i = 0$ and $B = 0$. Otherwise $\pi(K) =$ Unknown, which admits B. (c) A Done completion carries the reply, which (b) covers, read as a pair or as one effect. A NotDone completion of a call to a Ball means that the request was refused or never delivered; if it was the only request that could start the operation, no operation started and the composite made no call, which $(0, 0)$ admits. A call that was not made and can no longer be made has no effect either, and $(\bot, \bot)$ admits anything. The summand therefore satisfies the hypothesis of Proposition 1 in the caller, and induction on the depth of nesting completes the proof. □

The counting rule and the deferred reply both matter. A reply computed this way before

settlement is sound, but it says little, because the calls still to come are $\bot$. A reply that counted them as 0 would not be sound: a trip that answers at once that its hotel is not booked is contradicted when the hotel is booked a moment later. A deferred reply takes time, so a caller whose deadline is shorter than the composite's settlement ends its call Unknown, which is honest. Hypothesis (iii) of Corollary 1 matters as well. Under the Transient profile, a trip that crashes after its flight request was sent restarts with an empty ledger. If its caller sends the request again, the new trip books again and can settle with NotDone, for example after a sold-out answer, while the first seat is held. The projection π is sound but not exact: Partial, NotAllDone and SomeDone all become Unknown. A composite that can end partially should therefore reply with K. The caller then holds, by associativity, the knowledge of the flattened operation, to which Proposition 2 applies when all leaf calls are independent.

Transaction managers already merge and nest outcomes, more coarsely. Participants report heuristic outcomes [87]. The Narayana transaction manager, for example, merges one participant's heuristic commit and another's heuristic rollback into *mixed*, which absorbs later outcomes, and its interposed subordinate coordinator reports the merged outcome to its superior as one participant [62]. *Hazard* does not record which way the known participants went, so it covers NotAllDone, SomeDone and Unknown: merging hazard with a rollback stays hazard, whereas SomeDone + NotDone = Partial. In that implementation the merge also depends on arrival order: commit, hazard, rollback gives hazard, while commit, rollback, hazard gives mixed [62]. Here + is commutative, and both orders give Partial. Heuristic outcomes also signal a failure of atomicity, whereas the algebra describes operations that are not atomic, for which Partial is an ordinary result.

The kernel's `Knowledge` type implements the algebra. An exhaustive test checks these things:

- the connectives against supervaluation, which is the inductive step of the proof of Proposition 2;
- the monoid laws on all seven values;
- soundness and exactness for up to four constituents, each with at most one compensation (69,905 worlds), and for up to three constituents, each with up to two compensation attempts (127,551 worlds);
- that $\pi(K)$ admits the truth in every world.

The propositions hold for any number of constituents and compensation attempts; the test checks small cases directly. The helper `reportedKnowledge` lifts a reply into the caller's knowledge by the rule above. A further test checks it, including NotDone completions that the runtime itself produces, such as Overloaded, which count as Unknown unless the call was the only request.

Exactness is relative to the constituents' individual outcomes. If $\mathbf{K}e$ reads "it is known that e", conjunction matches epistemic logic, since $\mathbf{K}(a \wedge b) \Leftrightarrow \mathbf{K}a \wedge \mathbf{K}b$ [36], but disjunction does not. Evidence that correlates constituents, such as a log showing that one of two calls landed without saying which, can establish more than the algebra derives. The algebra under-approximates such knowledge and never over-approximates it.

Example. A trip *Flow*, a Ball whose state is a multi-step workflow, books a flight, which is NonIdempotent, and a hotel, which is Idempotent. When it cannot hold both, it voids or cancels what it booked; the providers then fence the trip's reference, so these compensations are absorbing. It makes at most two compensation attempts per booking. It answers the booking request only once settled, with its final K; a repeated request waits for the same answer, and beyond sixteen waiting requests the trip answers at once with an unsettled status, which a caller counts as Unknown. Under the Durable profile, an exhaustive exploration covers every combination of six fates per booking attempt, among them a request that reaches the provider late, three per compensation attempt, seat and room availability, and one crash before any of the 22 scheduler tasks of the longest run:

6,438 runs. In 3 of them the crash came before the request was accepted. The other 6,435 settled, with knowledge Done 2, NotDone 868, NotAllDone 2,977, Partial 172, SomeDone 379 and Unknown 2,037. In every run, the settled knowledge, and every status the trip held at every scheduler task, admitted the providers' real bookings. In 822 runs a booking request reached a provider after the trip's void or cancel, and the fence absorbed it. With the fence removed, the same exploration finds false settled claims in 115 of 7,257 settled runs.

Three controls show what the algebra adds. A status that counts every call without a completion, open or still to be made, as 0 (a report of the completions so far) was contradicted by the final bookings in 5,920 of the 6,438 runs. A two-valued status ("booked" if both bookings answered booked, otherwise "failed", read as nothing held) was wrong in 1,190 of the 6,435 settled runs, each of them a run in which the Flow gave up compensating after its two attempts. This count depends on the budget: without a crash, it is 30 of 85 settled runs with one attempt, 66 of 361 with two, and 138 of 1,489 with three. These are counts of enumerated paths, not rates. A saga-style status with a third value, "needs attention" for the 5,565 runs in which the Flow gave up, was never wrong. The algebra refines that value into NotAllDone 2,977, Unknown 2,037, SomeDone 379 and Partial 172, and it composes (below). An absorbing compensation that completes Done turns an Unknown booking into a definite NotDone, since $\bot \wedge \neg 1 = 0$; a compensation whose answer is lost leaves the booking unknown, and can make a held booking unknown. Of the 6,435 settled runs, the bookings alone left the knowledge uncertain in 6,311. Compensation made 1,002 of them definite and 84 others uncertain, which leaves 5,393. Compensation narrows ambiguity; it does not remove it.

The same exploration under the Transient profile measures what hypothesis (iii) of Corollary 1 excludes. A restarted trip is back at its start and claims nothing: its knowledge is Unknown, although in 635 runs a provider holds a booking that the trip forgot. If the caller then sends the request again, the new trip's settled claim is false in 608 of 3,073 settled runs, and the flight is booked twice in 152. The example therefore requires the Durable profile.

Nested composites. An itinerary Flow books an outbound and a return trip through the trip's port, with one request each, and adds the two settled replies with $+$. By associativity, its knowledge equals that of the four bookings taken together. Three explorations check it under the Durable profile:

- four fates per booking attempt, two per compensation attempt, and one crash before any of the 42 scheduler tasks of the longest run: 29,774 runs, of which 29,771 settled;
- every fate of the trip's exploration and seat and room availability, without a crash: 130,321 runs, every pair of the trip's 361 crash-free runs;
- the fates of the first exploration without a crash, and a deadline for each trip of 2,500 ms, shorter than the trip's worst case of 6,000 ms: 841 runs, in which 812 of the 1,682 calls to a trip ended Unknown.

In every run, the itinerary's settled knowledge admitted the four real bookings, and each trip's lifted reply admitted that trip's own bookings. Lifting each reply as a Boolean ("trip done") instead was wrong in 20,167 of the 29,771 settled runs of the first exploration and in 43,170 of the 130,321 runs of the second. No explored run produced a NotDone or an interim reply from a trip; unit tests cover those cases.

6 One Rule Set at Three Levels

The rules mention no language, transport or storage. Table 3 maps the concepts onto the three references; each reference also carries seeded bugs that its checks must catch. The references do not

Concept	Service (Kotlin)	Firmware (C11)	Hardware (SystemVerilog)
Owner (R1)	runtime-held state	one NVM record	registers with one writing process
Decider (R2)	pure functions	`step()` in its own unit	one latch-free `always_comb`
Commit horizon	one store write	CRC-checked slot write	the clock edge
Send horizon	write-ahead `Sending`	write-ahead record	`valid && ready` (AXI: VALID [5])
`Unknown`	timeout or restart while `Sending`	power loss while `Sending`	accepted transaction without response
Knowledge (R14)	`Knowledge` type	`ba_knowledge` type	status register, two bits per coordinate
Profile	Transient or Durable	Durable	Transient

Table 3: The same concepts at three levels.

check the same rules: the last column of Table 2 lists the levels at which each rule is checked. All three check the call-protocol rules R5–R8 and the algebra (R14).

Services. The Kotlin reference consists of the following parts:

- a dependency-free kernel of about 280 lines of code, including the knowledge algebra;
- a runtime of about 1,100 lines that implements both profiles, with an in-memory store for simulation, and a file store of about 300 lines that commits by atomic rename;
- a test kit whose oracles check R3 and R5–R9 on execution traces against simulated ground truth, under exhaustive or seeded exploration of network fates and crash points in the style of stateless model checking [31];
- architecture rules that check R1, R2 and R10 on bytecode.

The oracles are also defined over a language-neutral *conformance trace*. It holds one JSON object per line: the runtime's events (decision started and finished, commit, send attempt, resolution, reply, recovery) and, for each call handed to a simulated destination, a ground-truth line that says how many of its requests reached the destination, whether the destination processed it, and how often it applied the effect. Its properties T3 and T5–T9 are the checkable parts of R3 and R5–R9; R4 shows only through crash exploration, and R11 and R13 are not trace properties. A Python script that needs only the standard library checks the properties, so a runtime in another language can show its conformance by writing the format. The script accepts the 46 sample traces of the Kotlin explorations and clean runs and rejects each of 11 traces with a seeded violation; its messages match those of the Kotlin oracles, which name the rule (R7) where the script names the property (T7).

Evolution. A Durable store outlives the code that wrote it. Each record keeps the state version of the code that committed it, and each call the effect class it was committed with. On start, the runtime converts older states with the Ball type's pure `migrate` function, and it classifies every call by its recorded class (Section 4). Before it writes anything, it refuses to start, and names each offending record, if it cannot read a record or migrate its state, or if the new composition no longer registers a Ball type that has records, no longer binds the port of an open call, or no longer subscribes a Ball type to which a notice is still owed. A Ball type is retired by draining its records and then deleting them. Twelve tests check these cases, including a migration through the file store.

Firmware. The C reference commits state and the call ledger as one CRC-checked record in one of two NVM slots. It records each attempt ahead of transmission, so a power loss after the

write-ahead record turns a dose command into Unknown rather than a second dose. Time is a 64-bit count of milliseconds. There are two examples:

- an infusion-pump controller that gives a prescribed number of doses through a pump whose dose command has no idempotency key;
- a transfer that needs a source and a destination valve open. It opens one, then the other, and closes what it opened when it cannot have both; a close fences the open it undoes, so it is an absorbing compensation, and it is Idempotent, so the runtime retries it. The controller's state carries the transfer's knowledge, which a STATUS request returns.

A bounded exhaustive exploration places one fault at every power-loss point and NVM write of every run: a power loss, a failed write, or a torn write that keeps 4 bytes, 16 bytes or half of the record. It also explores the link fates of the first six transmissions of a dosing run and the first seven of a transfer run. That gives 937,229 dosing runs and 982,599 transfer runs. No violation of R4–R8 occurred, nor any overdose or dose spacing below the prescribed interval, nor any claim about the transfer, in committed state or in a STATUS reply, that the valves' final positions contradict. A deeper run explores 10,006,169 dosing runs over the first eight transmissions and 3,212,051 transfer runs over every transmission. The C algebra is checked exhaustively against enumerated worlds: in the same 69,905 worlds as the Kotlin test, and in 5,421,361 worlds for claims made before the operation ends. A purity lint reads the decider's object file, its preprocessed source and, with clang, its syntax tree. It refuses undefined symbols other than compiler helpers, writable sections, volatile accesses, inline assembly, impure builtins and casts to pointer types (R1, R2), and 25 negative controls must each be refused. It is necessary but not sufficient: pointer arithmetic from an argument, for example, passes. Forty-seven seeded bugs must each be found by the oracle named for it. Unlike the Kotlin runtime, the firmware records no effect class with each call and classifies an open call by its current port table; at start it compares only a fingerprint of the record's layout, instead of the checks described under *Evolution*. A firmware update that changes a port's effect class must therefore be applied with an empty call ledger, or must change the record's schema so that an older record quarantines the instance instead of being misread. Intermittent computing meets the same hazard on energy-harvesting devices (Section 9).

Hardware. The RTL reference has two blocks, and each of them is built the same way:

- its registers have one writer;
- its next-state logic is one latch-free `always_comb` block;
- its outputs depend on registered state only.

A command issuer issues READ (Safe), CONFIG (Idempotent: every attempt carries the command's id, by which the device deduplicates it) and FIRE (NonIdempotent) transactions on a bus with tagged responses. A composite block performs a two-step operation, CONFIG then FIRE, through the issuer. A status register holds the operation's knowledge, two bits per coordinate for the set of values it may take, so that Kleene's connectives become bitwise operations. Yosys checks single drivers; that flip-flops clocked on the rising edge of the clock input are the only storage; and that no path leads from an input to an output, or to an asynchronous flip-flop pin, except through a register. Each of 15 fixtures must be rejected for its named reason. Fault-injecting benches run against a device model: 8,764 issuer scenarios at each of three parameter sets, and 1,458 composite scenarios. The issuer bench checks the outcome kind that Table 1 requires for the attempts' fates, including Unknown, the deadline of each phase, and that FIRE is accepted and CONFIG applied at most once. The composite bench checks that every value of the status register admits the device's ground truth, and that the final value equals the knowledge computed by supervaluation. The blocks are Transient: the status register covers only the operations since the block's last reset, and the bench

resets the block and the device together, never during an operation. Twenty-eight seeded bugs must each fail a check. The first draft of the issuer reported NotDone when a CONFIG retry was withdrawn before acceptance, although the first attempt had already been applied; the bench caught it, and it is kept as a seeded bug. Hardware engineers apply such rules by experience. Stating them as rules lets a tool check them.

Closest work. Bluespec's guarded atomic actions and the Bluespec Codesign Language compile one rule-based semantics to both hardware and software [44, 65]. Lingua Franca reactors and synchronous languages give deterministic semantics across targets [10, 35, 52]. These unify how computation executes. Ballarch unifies what is known about external effects, and checks its call protocol and its algebra at each of the three levels, one level at a time.

7 Formal Model of the Call Protocol

An Alloy 6 model [41] covers the protocol of Table 1:

- commit, write-ahead, send, retry, deadline, crash and recovery;
- a lossy, duplicating network;
- the three effect classes.

Within bounds of three calls and traces of up to 14 steps, it has no counterexample to six safety properties: no send before commit, stable outcomes, no false NotDone, at most one send of a NonIdempotent call, Done only after processing, and agreement of state and call ledger. Under fairness, with two calls and up to 12 steps, every awaited call gets an outcome. Each of six seeded runtime bugs yields a counterexample, for example sending before the write-ahead record or re-sending after a crash. These are bounded results, not proofs for all sizes.

8 Evaluation

The benchmark compares nine implementations of one feature. It asks four questions:

RQ1. Under provider faults and crashes, which designs meet the correctness requirements, and under which assumptions?

RQ2. Where does each correct design keep the knowledge that makes it correct, and what does Ballarch cost in structure?

RQ3. Do the oracles and the crash cells detect seeded bugs, including bugs in recovery?

RQ4. How large is a change in each design, and is the design still correct after it?

The method, every table and the raw data are in the repository's `benchmark` directory [84].

Scenario. Three orders, all sent at time 0, compete for two units of stock. Each order reserves a unit, charges 100 through a payment provider, and completes; if the payment is not made, it releases the unit and fails. A database with atomic transactions survives crashes; everything else is lost. After a restart, the client sends again every order that the system has not durably accepted. The provider has a Safe status endpoint and declares a *delivery horizon* of 1 s: a request still unanswered 1 s after it was sent can no longer take effect. In *scenario A* the provider has no idempotency keys. In *scenario B* it honours an `Idempotency-Key` header [43] and does not charge a key twice. Oracles check seven requirements against the provider's ground truth: no double charge; no order reported failed while charged (a *false failure*) or completed while not charged; a terminal state for every accepted order; and no stock leak, no oversell and no charge for an order that the system never

accepted. A NeedsAttention terminal state, which hands the order to an operator, is allowed and is reported as a cost.

Faults and crashes. Each provider request draws one fate: answered in time; answered after 5 s; lost; processed with the answer lost; processed, then the connection reset; refused before sending; or delivered after the horizon, which takes effect 5 s after it was sent and is never answered. *Stress* draws the first six fates with equal probability. *Mild* answers 90% of requests in time and gives each of the other five fates 2%. *Beyond horizon* adds delivery after the horizon to Mild, at about 2%, and so breaks the provider's declared horizon. Each cell has 2,000 seeded runs, either without a crash or with exactly one. The crash time is drawn uniformly from the busy period of the same seed's crash-free run, from time 0 until every accepted order has its terminal state. The crash comes before every scheduler task due at that time, and the process restarts 50 ms later. Every design therefore gets the same distribution of crash times relative to its own run. Scheduler tasks take no virtual time, so a sampled crash never falls between two tasks of one instant. A separate sweep covers those points: for each of the first 1,000 seeds, it crashes the run once before each scheduler task of its crash-free run. The simulation is deterministic, and rerunning it reproduces the published data byte for byte.

Implementations. The author wrote nine implementations in Kotlin: four styles, each in a naive and a careful version, and Ballarch, with inventory, payment and checkout Balls on the reference runtime under the Durable profile. The styles are layered (Clean Architecture [56]), typed actors with `ask` and durable state [1, 39], MVU (The Elm Architecture [21, 22], with the model persisted before its commands run, an addition by the author), and hexagonal participants with a transactional outbox and an orchestrated saga [30, 73]. None of these sources says how to handle an ambiguous provider failure. The four naive versions therefore share one policy, chosen by the author: one attempt, and a timeout or I/O error means that the payment failed. Akka's documentation warns that the receiver of an `ask` that timed out may still process the request [1]. After a restart, each naive version sends again the charge of an order it was paying: the layered version resumes from the persisted status, the actors re-drive their current stage (an at-least-once addition by the author, since Akka's durable state does not re-run side effects), MVU's `init` issues the charge command again, and the saga's relay redelivers it.

The four careful versions handle the uncertainty with their own style's tools, without Ballarch's kernel or runtime. They apply the reconciliation policy of the specification's guidance, as Ballarch's payment Ball does. Each records durably that a charge may be in flight before sending it, and sends it again only after a refusal, with at most three requests. When the outcome is unknown, it waits past the horizon and queries the status, at most three times, then hands the order to an operator. After a restart, it reconciles a charge that may have been sent and never sends it again. The layered version, hardened by hand, saves a `CHARGING` status; the actors get a typed reply with an `Unknown` case, a persisted `Reconciling` stage and a timer; MVU gets a `Reconciling` state and a `CheckStatus` command; the saga's payment participant commits a charge-attempted row and can reply `PaymentUnknown`, which parks the saga. The budgets are the same in all five designs that reconcile. In Ballarch the payment Ball states the policy, and the runtime classifies each outcome and keeps the write-ahead phase and the recovery. Code that stands in for a framework (an actor toolkit, an Elm-style runtime, transactional messaging) is shared by the two versions of a style and, like Ballarch's runtime library, is excluded from the structural metrics.

In scenario B every design sends an idempotency key: `charge-<order>` in the eight others, the call's identity in Ballarch. The naive versions repeat an unanswered request with the same key,

		Stress		Mild		Beyond horizon	
	Design	none	one	none	one	none	one
A	Naive, each of the four styles	73.4	91.9	11.7	94.3	14.8	94.2
	Careful, each of the four styles	0.0	0.0	0.0	0.0	4.2	3.5
	Ballarch	0.0	0.0	0.0	0.0	4.2	3.5
B	Naive, same-key retries, then failed	80.6	70.9–71.0	0.1	0.2–0.4	0.2	0.1
	Layered, same-key retries, then NeedsAttention	0.0	0.0	0.0	0.0	0.0	0.0
	Careful, each of the four styles	0.0	0.0	0.0	0.0	0.0	3.3
	Ballarch	0.0	0.0	0.0	0.0	0.0	0.0
	Ballarch, reference adapter	0.0	0.0	0.0	0.0	4.2	1.9

Table 4: RQ1: percentage of runs, of 2,000 per cell, that violated at least one requirement, without a crash (none) and with one crash (one). Wilson 95% intervals have a half-width of at most 2.1 points; for 0.0 the upper bound is 0.2. Stress and Mild respect the provider's declared delivery horizon; Beyond horizon does not. In scenario A the four naive versions behave identically run for run, and so do the four careful versions and Ballarch without a crash. In scenario B, Ballarch's adapter ends each attempt after 1 s; the *reference adapter*, like the reference HTTP adapter, ends an attempt only at the call's deadline.

up to three requests, and then fail the payment; one more layered variant ends those retries in NeedsAttention instead. The careful versions repeat an unanswered or refused request within the same budget of three, then reconcile. Ballarch declares the charge port Idempotent. Its adapter ends each attempt after 1 s, and the runtime repeats a failed attempt with the same key, up to three requests in a call of 3 s; then the payment Ball reconciles. After a restart, the Durable runtime sends a possibly sent call again with its key while attempts and time are left, where the careful versions reconcile by status. Without crashes, Ballarch and the careful versions send the same requests and reach the same outcomes in every run.

RQ1: correctness. Table 4 gives the results.

The naive rates measure a policy, not the styles. Without crashes, a run violates a requirement when one of the two paid orders is charged but its answer reads as a failure. If p is the probability of such a fate, that happens with probability $1-(1-p)^2$: 0.75 under Stress ($p = 1/2$) and 0.116 under Mild ($p = 0.06$). With one crash, all four naive versions violated a requirement in 91.9% (Stress) and 94.3% (Mild) of runs, identically run for run: between two instants they hold the same durable state, and each recovery path sends the charge again. Under Mild, 1,866 of the 2,000 runs with a crash charged an order twice. The paths differ only when a crash falls between two tasks of one instant; the sweep shows that the naive actors version can then lose a fire-and-forget release and leak stock.

The careful versions and Ballarch are equally correct while the horizon holds. None of the five violated a requirement in its 8,000 Stress and Mild runs in either scenario, or at any of the 445,405 crash points swept under those profiles in scenario A (453,032 in scenario B). In scenario A they differ only in cost after a crash: 0.27–0.32 NeedsAttention per order under Stress (0.269 for the careful layered version, 0.296 for the careful saga, 0.304 for careful actors and MVU, 0.320 for Ballarch), and about 0.001 under Mild.

All five depend on the horizon. They treat a "not charged" status, read after the horizon, as final. Beyond the horizon in scenario A, all five reported charged orders as failed in 4.2% of crash-free runs and 3.5% of runs with a crash. In the sweep, 37–56 of each design's 1,000 seeds had a violating crash point (Ballarch 37). We compare the sweep by seeds, not by shares of crash points, because

Metric	Naive (range)	Careful				Ballarch
		Layered	Actors	MVU	Saga	
SLOC	96–184	216	221	153	231	279
JVM opcodes	1,504–2,242	2,854	3,247	2,945	3,034	3,729
Named classes (top-level; files)	10–18 (6–11; 2–6)	20 (12; 6)	24 (9; 3)	22 (9; 2)	21 (9; 5)	35 (16; 5)
Functions	9–20	26	13	15	14	32
Framework types	3–7	5	5	5	8	40
CBO, mean (max)	3.47–4.11 (9–14)	4.70 (16)	3.67 (23)	5.00 (21)	4.29 (15)	3.94 (18)
LCOM4, mean (at 1/measured)	1.00–1.67	1.00 (6/6)	1.67 (2/3)	1.00 (2/2)	1.00 (4/4)	1.60 (2/5)
CAMC, mean	0.53–0.77	0.66	0.52	0.37	0.67	0.40
D, mean: feature packages	0.34–0.47	0.35	0.44	0.40	0.46	0.44
D, mean: all dependencies	0.01–0.43	0.35	0.13	0.03	0.42	0.26
Propagation cost: named classes	20.7–31.0%	20.0%	14.8%	16.1%	18.6%	13.1%
top-level classes	28.1–41.7%	26.4%	30.9%	32.1%	29.6%	23.4%
source files	48.0–75.0%	50.0%	66.7%	75.0%	48.0%	48.0%
Cyclomatic complexity, mean (max)	2.10–3.44 (7–13)	2.23 (7)	4.77 (31)	3.73 (15)	3.50 (18)	2.50 (8)
Cyclomatic complexity, total	31–42	58	62	56	49	80
Exception handlers	1–3	5	3	5	5	0
Failures returned as values	0–2	0	4	0	0	6
Code in pure classes	26–61%	28%	28%	66%	40%	89%

Table 5: RQ2: structure of the feature code in scenario A; *Naive* gives the range over the four naive versions. Metrics: CBO [19]; LCOM4 [40] and CAMC [7] over hand-written methods, without the members the compiler generates; distance from the main sequence D [55], counting efferent coupling to other feature packages only, or to every dependency except the JDK and Kotlin's standard library as Martin defines it; propagation cost [53] at three granularities; cyclomatic complexity [57] counted on bytecode as JaCoCo defines it [42], without the branches the compiler generates. Exception handlers are `catch` clauses and `runCatching` calls. *Failures returned as values* counts the calls whose failure or timeout a library returns as a value: `ask` and `pipeToSelf` in the actor toolkit, `Call` and `Runtime.request` in Ballarch. A pure class has no mutable field and no reference to concurrency, I/O, clocks, randomness, the database, the provider or a runtime.

each design splits its work into scheduler tasks differently.

Idempotency keys (scenario B). No design charged an order twice. Naive same-key retries did not become correct: after three unanswered requests, one of them has usually taken effect, so 80.6% of crash-free Stress runs reported a charged order as failed. Ending the same retries in NeedsAttention was correct in every cell, at 0.394 operator hand-offs per order under Stress without a crash, against 0.221 for the reconciling designs (0.310 against 0.228–0.248 with one crash). With the same budget, Ballarch violated no requirement in any cell. The careful versions violated one only beyond the horizon with a crash, in 3.3% of runs (66 of 2,000 for each), because after a restart they reconcile by status where Ballarch's runtime sends the call again with its key. A careful version could also send again with the key on recovery; that variant was not measured. In the sweep, 1 of Ballarch's 1,000 seeds had a violating Beyond-horizon crash point (5 points), against 37–38 seeds for each careful version. Ballarch's zero needs an adapter that ends each attempt itself. With the reference adapter, whose attempt ends only at the call's deadline, the runtime never repeats a request whose answer was lost, and Ballarch reported charged orders as failed beyond the horizon in 4.2% of the runs without a crash and 1.9% of those with one.

RQ2: where the care lives, and what it costs. Table 5 compares the structures. Each careful version is 47–58 SLOC larger than its naive version, and its total cyclomatic complexity is 12–25

higher (from 42 to 58 for layered, 40 to 62 for actors, 31 to 56 for MVU, 37 to 49 for the saga). That is the price of writing the care into the feature. Ballarch keeps most of it elsewhere; its payment Ball still states the reconciliation. Its feature code has no exception handler, against 3–5 in the careful versions; instead it makes 6 calls whose failure or timeout the runtime returns as an `Outcome` value (careful actors make 4 such calls to the actor toolkit, the other careful versions none). Its two adapters classify provider failures once, as values of the provider's callback API, where the careful versions classify them in `catch` clauses around a throwing client, so the handler count reflects that API as well as the architecture. The write-ahead record, the classification of each attempt and the recovery after a restart are in the runtime, not in the feature; RQ3 shows that a recovery bug then lives in the runtime as well. 89% of Ballarch's code is in pure classes, against 28–66% in the careful versions. Composite operations get the same treatment: the trip and the itinerary of Section 5 derive their claims from the algebra, and those claims admitted the real bookings in every explored run, where a two-valued status was wrong in 1,190 of 6,435 settled trip runs. A saga-style status with a third value, "needs attention", was never wrong either; the algebra splits that value into four and composes, as the itinerary shows.

These gains have costs. Against the careful versions, Ballarch has more SLOC (279 against 153–231, 21% more than the largest) and more bytecode (3,729 opcodes against 2,854–3,247), 40 framework types against 5–8, more named classes (35 against 20–24) and functions (32 against 13–26), and the highest total cyclomatic complexity of the nine designs (80 against 49–62). Its cohesion is lower than that of the careful layered, actors and saga versions by CAMC (0.40 against 0.52–0.67; careful MVU 0.37), and lower than that of the careful layered, MVU and saga versions by mean LCOM4 (1.60, with 2 of 5 measured classes at 1, against 1.00; careful actors 1.67). LCOM4 counts a decider that keeps its state in a parameter rather than in a field as split. Ballarch's mean distance from the main sequence is 0.44 (careful versions 0.35–0.46) when only dependencies between feature packages count, and 0.26 (0.03–0.42) with Martin's full efferent coupling. In exchange its functions are smaller: a mean cyclomatic complexity of 2.50 and a maximum of 8, against 3.50–4.77 and 15–31 for three careful versions (the careful layered version is lower, at 2.23 and 7). Its mean CBO is lower than that of three careful versions (3.94 against 4.29–5.00; careful actors 3.67). Its propagation cost is the lowest of the nine per named class (13.1% against 14.8–31.0%) and per top-level class (23.4% against 26.4–41.7%). Per source file, the granularity that MacCormack et al. used, it ties with both saga versions at 48.0% (the others 50.0–75.0%), on graphs of 2 to 6 files.

RQ3: seeded bugs. Seven plausible bugs were injected from outside the correct implementations and run in scenario A under Stress and Mild, where the correct versions have no violation. Two misclassify an uncertain charge: a Ballarch payment Ball that treats `Unknown` as not charged, and a hardened gateway that reports an uncertain charge as not sent. Both were detected without crashes (in 92.3% of Stress and 12.4% of Mild runs) and with one (Ballarch's in 96.3% and 94.5%, the hardened one in 48.2% and 3.1%). The other five break only recovery, by rewriting what a restarted process finds in its durable records. Ballarch's store reads a possibly sent charge as never sent, so the runtime sends it again; the hardened recovery resumes `CHARGING` orders as `RESERVED`; careful actors resume a `Charging` checkout from `Reserving`; careful MVU loses from its saved model the orders still being paid; and the careful saga loses its charge-attempted rows. Each was detected in runs with a crash and in none of the 2,000 crash-free runs per profile: Ballarch's runtime bug in 13.3% (Stress) and 85.7% (Mild) of runs, the hardened one in 70.0% and 93.6%, and the other three in 15.2% and 85.8% each. The crash cells therefore reach the state that each recovery path exists for, and the zero rates of RQ1 depend on that path. In Ballarch the recovery under test is the runtime's; in the careful versions it is feature code.

	Code lines			Tokens		Ballarch's	Files; units	
Change request	Ballarch	careful	naive	Ballarch	careful	rank	Ballarch	careful
CR1 refund	73	34–43	33–41	1,140	341–553	5/5/5/5	4; 16	2–5; 5–6
CR2 loyalty points	50	64–68	27–37	899	633–974	1/4/1/4	4; 9	2–5; 6–8
CR3 failover	18	11–16	7–12	249	104–179	5/5/5/5	2; 5	2–4; 3–6
CR4 no inventory	38	8–48	8–41	720	109–584	3/3=/4/5	4; 9	2–5; 3–7
All four	179	122–173	77–126	3,008	1,527–1,954	5/5/5/5		

Table 6: RQ4: size of each change. Code lines are logical lines (statements, declaration headers and `when` branches, however they are wrapped) added, removed or changed, without import lines; *careful* and *naive* give the range over the four versions. Ballarch's rank among the five reconciling designs runs from 1 (smallest) to 5 (largest), by code lines, code lines with import lines, physical lines and tokens; = marks a tie. Units are top-level classes, interfaces or objects, or the top-level declarations of one file.

RQ4: change impact. Four change requests were specified in prose before any design was changed, then applied to all nine designs as patches. CR1 refunds an order cancelled within 30 s of its acceptance. CR2 awards loyalty points through a second service that, like the scenario-A provider, has no idempotency keys. CR3 fails over to a second provider when the first provably did not charge. CR4 removes the inventory. With each change, the careful versions and Ballarch violated no requirement, old or new, in any of their 8,000 Stress and Mild runs per change request, or at any of 25,305–96,455 swept crash points per design and change request. Without the change, every design violated the change request's own requirements. Beyond the horizon, the reconciling designs awarded points twice in 170–187 of 4,000 runs in CR2 and charged twice in 150–151 runs in CR3.

Ballarch's changes were larger in total by every measure (Table 6): 179 code lines against 122–173 for the careful versions, 242 against 131–192 with import lines, and 3,008 tokens against 1,527–1,954. They were the largest of the five reconciling designs in CR1 and CR3 by every measure. In CR1 both the checkout and the payment Ball had to accept a second request type, and the acceptance time had to pass through three states of the checkout. In CR2, a second effect without idempotency keys, Ballarch needed the fewest code lines, because the write-ahead record, the classification of each attempt and the recovery came from its runtime; every reconciling design, Ballarch included, still wrote the award's reconciliation. With import lines or by tokens it ranked fourth of five. In CR4 it was within the careful versions' range by lines and the largest by tokens. It touched 2–4 files per change, within the careful versions' 2–5, and more top-level units in three of the four. Adapting each design to idempotency keys (scenario B) also changed the most lines in Ballarch: 29 lines added or changed in 2 files, 19 of them in the charge adapter, mostly for its attempt timer, against 2–10 for the careful versions.

Runtime overhead. On a virtual machine with 4 processors, Linux and OpenJDK 21, a request (a counter increment) through the reference runtime on its production thread scheduler, with an in-memory store, took a median of 46.0 µs (99th percentile 121 µs; the medians of three rounds were 45.1–46.0 µs). A direct call of the same decider took 0.04 µs. With its tasks run on the caller's thread, the runtime's own work took a median of 0.68 µs and allocated about 1,144 bytes per request, so most of the difference is the hand-off between threads. The same-thread and direct times were still falling from round to round, so they are upper bounds for a settled JVM; the allocation is not. With the durable file store, which forces every commit to the device, the median was 1.25 ms. The numbers depend on the machine and are not part of the deterministic data.

9 Related Work

Architecture styles. Architecture research describes styles as components, connectors and constraints [69, 75], and modules by the decisions they hide [67].

- Layered, Clean and Hexagonal architectures and domain-driven design [20, 27, 56] constrain dependencies.
- Actor systems give each piece of state one owner [6, 17, 39].
- The Elm architecture and the Decider pattern separate pure transitions from effects [18, 22].
- Durable-execution systems persist the progress of orchestrations and replay them deterministically [15].

Ballarch keeps the structural constraints of these styles. To them it adds the temporal boundaries, the outcome rules and the algebra. Meng and Jackson's structural pattern for legible software is a recent example of a new structural pattern evaluated on an application [58].

Ambiguous outcomes and idempotence. Several lines of work recognise ambiguous outcomes:

- the third outcome of RPC, a failed call that may or may not have been executed [11];
- Jepsen's indeterminate operations [45, 46];
- the heuristic outcomes of XA [87], including mixed and hazard;
- Helland's analysis of activities between partners [37, 38].

X-ability defines exactly-once correctness for services with side effects and is composable [29]. Ramalingam and Vaswani make workflows idempotent and therefore safe to retry [71]. These approaches hide or remove the third outcome. Work on LLM agents reports duplicated and hidden side effects at tool boundaries. Trofimov and Novikov separate effects in the world from what the agent runtime observes of them, and find that the effect fields of shared agent–tool interfaces give only coarse per-call hints [85]; the Model Context Protocol, for example, calls its tool annotations hints, on which clients should never base decisions when the server is untrusted [59]. Li shows that no policy that only reads back the outcome is exactly-once under late commits without a bound on in-flight time [51]; that bound is the delivery horizon on which reconciliation depends here (Section 8).

Retry rules in transports and runtimes. Transport protocols already apply send-horizon reasoning to single requests. HTTP says that a client should not automatically retry a request with a non-idempotent method unless it knows that the request is in fact idempotent, or can detect that the original request was never applied [28, Sec. 9.2.2]. HTTP/2 supplies such evidence: a GOAWAY frame's last stream identifier and the REFUSED_STREAM error code guarantee that a request was not processed, and a client may then retry it even if it is not idempotent [83, Sec. 8.7]. HTTP/3 defines a rejected request in the same way [12, Sec. 4.1.1]. gRPC retries transparently only while a call has not left the client or has not reached the server's application logic [25]; it calls an RPC *committed* when it stops making new attempts, an instant unrelated to our commit horizon. Service-design guidance commits local state before each foreign mutation, retries only mutations that are idempotent or carry a key, and marks a timed-out non-idempotent one as failed [50]. Durable-execution engines run activities under a retry policy, with unlimited attempts by default in Temporal; an attempt that times out after dispatch is retried if the policy allows it [81, 82]. Temporal records which timeout ended an attempt, so a workflow can tell an attempt that never reached a worker (Schedule-To-Start) from one that may have run (Start-To-Close) [80, 82]. In Temporal, mapping that to an outcome, and not retrying a non-idempotent activity, are left to workflow code and retry-policy configuration. Newer engines persist the send horizon itself. In AWS Lambda durable functions, a step declared `AT_MOST_ONCE_PER_RETRY` is checkpointed before

its code runs; if it is interrupted after that checkpoint, it runs again only on a new attempt of its retry strategy, and otherwise ends in an interruption error [2]. Golem, with its idempotence mode switched off, fails the worker instead of repeating a request whose delivery it cannot determine [32]. Both are chosen per step or scope by configuration, and both report an error.

These rules govern the retries of one protocol, client, service or step, and report the rest as errors or leave them to configuration. Ballarch makes them one contract of the architecture. The runtime derives each outcome, and whether it may retry, from the effect class declared for the port; it never retries a NonIdempotent call (R8), and refuses a retry policy for a NonIdempotent port when the composition is built. The outcome is a typed value delivered once to the owner's committed state, Unknown is kept apart from failure, under the Durable profile the phase survives crashes through the write-ahead record, the result composes through the algebra of Section 5, and the same rule is checked in firmware and RTL. Ballarch is also more conservative than HTTP/2: evidence of non-application ends the call as NotDone, and the decider, not the transport, decides whether to send it again.

Transactions and fault-tolerant runtimes. Transaction processing defers real actions until commit [33], and a participant in atomic commitment is uncertain of the outcome between its vote and the decision [9]; the two horizons carry these instants from transactions to every call of an application. Reliable State Machines are actor-like state machines whose state survives failures, with a proof of failure transparency [61]; they are the closest runtime model to Balls. Restate, a deployed durable-execution engine, comes close as well with its virtual objects: each has keyed state, at most one handler that may write it runs at a time, and the state is committed atomically with the handler's execution [72]. Such systems aim at exactly-once processing inside their boundary. Ballarch addresses destinations outside any such boundary, which may offer no key: it turns the remaining uncertainty into a typed outcome and composes it. The end-to-end argument [74] places duplicate suppression and delivery assurance at the endpoints. The Idempotent class puts deduplication at the destination, and the outcome rules let the caller state only what it can know.

Compensation and three-valued reasoning. Sagas [30] and compensation calculi [13, 16] give semantics to long-running transactions, but assume that outcomes are known. Try-Confirm/Cancel [66] does not: its implementations fence a request whose cancel arrived first ("empty rollback" and "anti-suspension" [3]), which is our absorbing compensation. Transaction managers merge heuristic outcomes and pass them up through interposed coordinators [62]. They thus say what a composite may claim while some outcomes stay unknown, but coarsely, depending on arrival order, and only for atomic transactions; Section 5 shows how the (*all*, *any*) algebra refines that merge. None of these approaches counts calls still to be made, or compensations, in such a claim. Three-valued abstractions and may/must approximations [14, 49] use the monotonicity argument of our soundness proposition, and ternary circuit simulation [24] uses the same connectives; its pessimism under reconvergent fanout is why exactness needs read-once formulas. Knowledge-based analyses of commitment [34, 36] reason about what processes know. The algebra here is truth-functional by design, so that a runtime can compute it.

Output commit and hardware. The output-commit problem [26, 78], write-ahead logging [60], external synchrony [63, 64] and precise interrupts [76] are the mechanisms behind the horizons. Intermittent computing meets the firmware reference's hazard on energy-harvesting devices: repeating I/O after a power failure causes idempotence bugs [79], peripherals need just-in-time checkpoints [54], and EaseIO lets programmers annotate each I/O call with its re-execution semantics [88]. Those

annotations are the closest firmware analogue of effect classes. These systems aim to make re-execution correct; Ballarch instead never re-executes a NonIdempotent command whose fate is unknown, reports Unknown, and applies the same rule at the other levels. The time-triggered architecture [48] makes time part of interfaces. Bluespec, BCL, Lingua Franca and synchronous languages [10, 35, 44, 52, 65] unify execution semantics across hardware and software.

10 Limitations and Threats to Validity

- **Construct and internal validity.** One author designed Ballarch and wrote all nine implementations of the benchmark, their adaptations to idempotency keys and every change-request patch. Each is one reading of its style. The naive versions share a failure policy that no cited source prescribes, so their rates measure that policy and the fault mix, not the styles. The careful versions apply the reconciliation policy of Ballarch's own specification, with Ballarch's budgets, and were written after the first results were known. They show that the policy can be written in each style without the runtime, not how independent experts would write it. A reviewer showed that the first adaptation of Ballarch to idempotency keys gave it a smaller retry budget than the careful versions, and it was changed. Parts of the implementations and patches were drafted with an AI assistant under the author's direction, which may make the designs more alike than those of independent authors. Independent implementations written before the results are seen would make the comparison stronger. The benchmark was not pre-registered.

- **External validity.**
  - The benchmark has one small scenario family: three orders for one product with two units in stock, a simulated provider and database, and at most one crash per run. Scenario B's provider keeps every key for the whole run.
  - The fault mix, the retry budgets, the provider's 1 s delivery horizon and the 5 s delay beyond it are assumptions, and the reconciling designs depend on the horizon. A sampled crash falls at a time uniform over its run's busy period, never between two tasks of one instant; the sweep covers those points but counts them per task.
  - The scenario-B adaptations are choices: the careful versions keep their scenario-A recovery, which reconciles by status, and Ballarch's adapter ends each attempt after 1 s, which the reference HTTP adapter does not.
  - No durable-execution engine, such as Temporal, Restate, Azure Durable Functions or AWS Lambda durable functions, is among the designs.
  - The four change requests are few. Three concern an unreliable external service, where Ballarch's runtime is meant to help, and none changes only business rules or data. Code lines and units touched are proxies for the effort of a change; developer effort and review time were not measured. The patches change only the scenario-A designs; the scenario-B variants would need further edits, which are not in the counts.
  - The firmware reference has two examples and the RTL reference two blocks, each checked in simulation against a model of its device; the RTL blocks are also checked on their netlists. The checks are bounded: by default, the firmware explorations place one fault per run and choose the link fates of the first six or seven transmissions, and the RTL benches combine nine fates for the first two attempts of a command. These references check the call-protocol rules and the algebra, not every rule (Table 2).
  - The algebra is checked at each level, and at the service level also for a composite of composites (the itinerary), but no reference composes levels, for instance firmware that drives the RTL

block as a Ball behind a port. The closure corollary covers such composition, but no check exercises it.
  - The examples that check the algebra by exploration are small: two bookings, two trips of two bookings, two valves and a two-step hardware operation.

- **Runtime.**
  - Claims about effects that outlive a crash, such as bookings held by a provider, need the Durable profile: under the Transient profile, the algebra covers only the calls made since the last start.
  - The runtime has no per-attempt timeout. An attempt ends when its adapter reports; otherwise the call's deadline ends the call. Retries of a Safe or Idempotent port therefore help only if the adapter ends each attempt itself, for example with its own timeout. The reference HTTP adapter does not, and Section 8 measures what that costs.
  - At start, the runtime checks that the new composition still registers every Ball type that has records, binds every port with an open call and subscribes every Ball type still owed a notice, and that every state can be read and migrated. It does not check whether a new decider still handles the completions of a port that is still bound; such a completion quarantines the instance when it arrives. Nothing prevents two runtimes from owning one instance during a rolling deploy; the handover is left to the deployment.
  - The file store is single-node and has been tested against killed processes, not power loss; it is not a production database.
  - The firmware reference records no effect class with its calls and checks only a layout fingerprint at start, so a firmware update that changes a port's effect class needs an empty call ledger or a new record schema.

- **Metrics.** LCOM4 and CAMC were designed for classes that share fields, and only 2 to 6 classes per design are measured. Propagation cost and CBO depend on how finely code is divided into classes: Ballarch's sealed state hierarchies give it 35 named classes in 16 top-level classes and 5 files. Martin's D depends on which dependencies count. Cyclomatic complexity on bytecode counts null-safety operators, comparisons used as values and the branches of some inlined standard-library functions, but not `catch` clauses. The failure-handling counts depend on the API each design calls. The runtime overhead was measured on one machine.

- **Formal results.** The Alloy results are bounded. The algebra's propositions and the closure corollary are proved, and the propositions are also checked exhaustively for small sizes. The closure corollary assumes honest adapters, absorbing compensations, a call ledger kept for as long as the calls' effects persist (the Durable profile, or crashes that also clear those effects), and deferred replies; the first two are reviewed, not verified.

11 Conclusion

Ballarch places two instants that mechanisms in recovery, operating systems and processors already respect into the architecture itself, as boundaries in time. A runtime derives each outcome from them, and a small algebra carries honesty from parts to wholes that reply with settled knowledge. The call-protocol rules and the algebra can be checked by level-specific mechanisms from register-transfer logic to services, though composition across those levels remains to be tested. In our benchmark, careful versions of four styles and Ballarch were equally correct while the provider's declared delivery horizon held, and all of them depended on it. Ballarch did not make the feature more correct under

that horizon. It moved the classification of outcomes, the write-ahead record and the recovery from the feature into the runtime and the rules, though the feature still states its reconciliation, at the price of more code, classes and framework types, lower cohesion, a higher total complexity, larger changes and a thread hand-off per request. Whether that trade pays off for teams, and for coding agents that write such features, is the next question to measure.

Artifacts. The specification, the Kotlin, C and SystemVerilog references, the Alloy model, the conformance-trace format (`tools/TRACE-FORMAT.md`) with its checker, the benchmark and its raw data are available in the Ballarch repository, `https://github.com/4wl2d/Pokeball`, which still carries the project's earlier name (version 2.0.0-rc.1). The file `docs/problems.md` there maps twelve recurring failures of current architectures to the mechanism, rule and evidence for each, and lists what Ballarch does not solve. Text is licensed under CC BY 4.0; the code has no software license yet.

Use of generative AI. The author used an AI assistant (Anthropic's Claude) to draft parts of the code, the benchmark and the text, under the author's direction. The author reviewed the material, ran the reported checks, and takes full responsibility for the content.

References

[1] Akka project. Akka documentation: *Interaction Patterns* and *Durable State*. `https://github.com/akka/akka`, 2026. Sections "Request-Response with ask between two actors" and "Side effects ordering and guarantees"; files akka-docs/src/main/paradox/typed/interaction-patterns.md and akka-docs/src/main/paradox/typed/durable-state/persistence.md, main branch, accessed 2026-10-21.

[2] Amazon Web Services. AWS Lambda durable execution SDK for Java: Steps. `https://github.com/aws/aws-durable-execution-sdk-java/blob/main/docs/core/steps.md`, 2026. Step semantics AT_LEAST_ONCE_PER_RETRY and AT_MOST_ONCE_PER_RETRY (checkpoint before execution; StepInterruptedException when retries are exhausted). Lambda durable functions generally available since 2025-12-02; accessed 2026-10-21.

[3] Apache Seata. Alibaba Seata resolves idempotence, dangling, and empty rollback issues in TCC mode. `https://seata.apache.org/blog/seata-tcc-fence/`, 2026. Describes the TCC fence (table tcc_fence_log) introduced in Seata 1.5.1; undated page, accessed 2026-10-21.

[4] Arm Limited. *Arm Architecture Reference Manual for A-profile Architecture*. Arm Limited, issue l.b edition, 2025. URL `https://developer.arm.com/documentation/ddi0487/latest`.

[5] Arm Limited. *AMBA AXI Protocol Specification*. Arm Limited, issue l edition, 2025. URL `https://developer.arm.com/documentation/ihi0022/latest`.

[6] Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, The Royal Institute of Technology (KTH), Stockholm, Sweden, 2003.

[7] Jagdish Bansiya, Letha Etzkorn, Carl Davis, and Wei Li. A class cohesion metric for object-oriented designs. *Journal of Object-Oriented Programming*, 11(8):47–52, 1999.

[8] Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4), 2011. doi: 10.1145/2000799.2000800. Article 14.

[9] Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987. ISBN 0-201-10715-5.

[10] Gérard Berry and Georges Gonthier. The Esterel synchronous programming language: Design, semantics, implementation. *Science of Computer Programming*, 19(2):87–152, 1992. doi: 10.1016/0167-6423(92)90005-V.

[11] Andrew D. Birrell and Bruce Jay Nelson. Implementing remote procedure calls. *ACM Transactions on Computer Systems*, 2(1):39–59, 1984. doi: 10.1145/2080.357392.

[12] Mike Bishop. HTTP/3. RFC 9114, June 2022. URL `https://www.rfc-editor.org/info/rfc9114`. Section 4.1.1 checked against the text at https://httpwg.org/specs/rfc9114.html.

[13] Roberto Bruni, Hernán C. Melgratti, and Ugo Montanari. Theoretical foundations for compensations in flow composition languages. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '05)*, pages 209–220. ACM, 2005. doi: 10.1145/1040305.1040323.

[14] Glenn Bruns and Patrice Godefroid. Model checking partial state spaces with 3-valued temporal logics. In *Computer Aided Verification (CAV '99)*, volume 1633 of *Lecture Notes in Computer Science*, pages 274–287. Springer, 1999. doi: 10.1007/3-540-48683-6_25.

[15] Sebastian Burckhardt, Chris Gillum, David Justo, Konstantinos Kallas, Connor McMahon, and Christopher S. Meiklejohn. Durable functions: Semantics for stateful serverless. *Proceedings of the ACM on Programming Languages*, 5(OOPSLA):133:1–133:27, 2021. doi: 10.1145/3485510.

[16] Michael Butler, C. A. R. Hoare, and Carla Ferreira. A trace semantics for long-running transactions. In *Communicating Sequential Processes: The First 25 Years*, volume 3525 of *Lecture Notes in Computer Science*, pages 133–150. Springer, 2005. doi: 10.1007/11423348_8.

[17] Sergey Bykov, Alan Geller, Gabriel Kliot, James R. Larus, Ravi Pandya, and Jorgen Thelin. Orleans: Cloud computing for everyone. In *Proceedings of the 2nd ACM Symposium on Cloud Computing (SoCC '11)*. ACM, 2011. doi: 10.1145/2038916.2038932.

[18] Jérémie Chassaing. Functional event sourcing decider. `https://thinkbeforecoding.com/post/2021/12/17/functional-event-sourcing-decider`, December 2021. Blog post, Accessed 2026-10-21.

[19] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994. doi: 10.1109/32.295895.

[20] Alistair Cockburn. Hexagonal architecture. `https://alistair.cockburn.us/hexagonal-architecture/`, 2005. Accessed 2026-10-21.

[21] Evan Czaplicki. A farewell to FRP: Making signals unnecessary with The Elm Architecture. `https://elm-lang.org/news/farewell-to-frp`, May 2016. Elm 0.17 release announcement, dated 2016-05-10; checked against pages/news/farewell-to-frp.elm in github.com/elm/elm-lang.org, accessed 2026-10-21.

[22] Evan Czaplicki. An introduction to Elm: The Elm architecture; commands and subscriptions. `https://guide.elm-lang.org/architecture/`, 2026. Checked against the guide's source, github.com/evancz/guide.elm-lang.org (book/architecture and book/effects), accessed 2026-10-21.

[23] Wei Dong, Martin Leucker, and Christian Schallhart. Impartial anticipation in runtime-verification. In *Automated Technology for Verification and Analysis (ATVA 2008)*, volume 5311 of *Lecture Notes in Computer Science*, pages 386–396. Springer, 2008. doi: 10.1007/978-3-540-88387-6_33.

[24] Edward B. Eichelberger. Hazard detection in combinational and sequential switching circuits. *IBM Journal of Research and Development*, 9(2):90–99, 1965. doi: 10.1147/rd.92.0090.

[25] Noah Eisen and Eric Gribkoff. gRPC retry design (gRFC A6). `https://github.com/grpc/proposal/blob/master/A6-client-retries.md`, 2024. Last updated 2024-08-23; section "Transparent Retries", accessed 2026-10-21.

[26] E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002. doi: 10.1145/568522.568525.

[27] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003. ISBN 0-321-12521-5.

[28] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP semantics. RFC 9110, STD 97, June 2022. URL `https://www.rfc-editor.org/info/rfc9110`.

[29] Svend Frølund and Rachid Guerraoui. X-ability: A theory of replication. *Distributed Computing*, 14(4): 231–249, 2001. doi: 10.1007/s004460100065.

[30] Hector Garcia-Molina and Kenneth Salem. Sagas. In Umeshwar Dayal and Irving L. Traiger, editors, *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, San Francisco, CA, USA, May 27–29, 1987*, pages 249–259. ACM Press, 1987. doi: 10.1145/38713.38742.

[31] Patrice Godefroid. Model checking for programming languages using VeriSoft. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*, pages 174–186. ACM, 1997. doi: 10.1145/263699.263717.

[32] Golem Cloud. Golem documentation: Durability. `https://learn.golem.cloud/develop/durability`, 2026. Idempotence mode: HTTP requests are assumed idempotent by default; with the mode off, a request whose delivery cannot be determined fails the worker. Checked on 2026-10-21 through search-engine extracts of the page, which could not be reached from here, and golemcloud/golem pull requests 4198 and 4207 (merged 2026-10-16), which keep the refusal for requests that cannot be re-executed and remove the global mode from the main branch.

[33] Jim Gray. The transaction concept: Virtues and limitations. In *Proceedings of the Seventh International Conference on Very Large Data Bases (VLDB), Cannes, France, September 1981*, pages 144–154, 1981. Also Tandem Technical Report TR 81.3; pages checked against the VLDB 1981 proceedings index.

[34] Vassos Hadzilacos. A knowledge theoretic analysis of atomic commitment protocols. In *Proceedings of the Sixth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS '87)*, pages 129–134. ACM, 1987. doi: 10.1145/28659.28672.

[35] Nicolas Halbwachs, Paul Caspi, Pascal Raymond, and Daniel Pilaud. The synchronous data flow programming language LUSTRE. *Proceedings of the IEEE*, 79(9):1305–1320, 1991. doi: 10.1109/5.97300.

[36] Joseph Y. Halpern and Yoram Moses. Knowledge and common knowledge in a distributed environment. *Journal of the ACM*, 37(3):549–587, 1990. doi: 10.1145/79147.79161.

[37] Pat Helland. Life beyond distributed transactions: an apostate's opinion. In *Third Biennial Conference on Innovative Data Systems Research (CIDR 2007), Asilomar, CA, USA, January 7–10, 2007, Online Proceedings*, pages 132–141. www.cidrdb.org, 2007. URL `http://cidrdb.org/cidr2007/papers/cidr07p15.pdf`.

[38] Pat Helland and David Campbell. Building on quicksand. In *Fourth Biennial Conference on Innovative Data Systems Research (CIDR 2009), Asilomar, CA, USA, January 4–7, 2009, Online Proceedings*. www.cidrdb.org, 2009.

[39] Carl Hewitt, Peter Boehler Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In Nils J. Nilsson, editor, *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI), Stanford, CA, USA, August 20–23, 1973*, pages 235–245. William Kaufmann, 1973. URL `https://www.ijcai.org/Proceedings/73/Papers/027B.pdf`.

[40] Martin Hitz and Behzad Montazeri. Measuring coupling and cohesion in object-oriented systems. In *Proceedings of the International Symposium on Applied Corporate Computing (ISACC '95)*, Monterrey, Mexico, 1995.

[41] Daniel Jackson. *Software Abstractions: Logic, Language, and Analysis*. MIT Press, Cambridge, MA, revised edition, 2012. ISBN 978-0-262-01715-2.

[42] JaCoCo project. JaCoCo documentation: Coverage counters, section "cyclomatic complexity". `https://www.jacoco.org/jacoco/trunk/doc/counters.html`, 2026. Filters of version 0.8.15; text checked against org.jacoco.doc/docroot/doc/counters.html in github.com/jacoco/jacoco, accessed 2026-10-21.

[43] Jayadeba Jena, Sanjay Dalal, and Dmitrii Tishkin. The Idempotency-Key HTTP header field. Internet-Draft draft-ietf-httpapi-idempotency-key-header, IETF HTTPAPI Working Group, 2026. Work in progress; editor's copy in github.com/ietf-wg-httpapi/idempotency, accessed 2026-10-21.

[44] Myron King, Nirav Dave, and Arvind. Automatic generation of hardware/software interfaces. In *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XVII)*, pages 325–336. ACM, 2012. doi: 10.1145/2150976.2151011.

[45] Kyle Kingsbury. Jepsen: Distributed systems safety research. `https://jepsen.io`, 2026. Accessed 2026-10-21.

[46] Kyle Kingsbury and Peter Alvaro. Elle: Inferring isolation anomalies from experimental observations. *Proceedings of the VLDB Endowment*, 14(3):268–280, 2020. doi: 10.14778/3430915.3430918.

[47] Stephen Cole Kleene. *Introduction to Metamathematics*. North-Holland, Amsterdam, 1952. Co-published with P. Noordhoff (Groningen) and D. Van Nostrand (New York).

[48] Hermann Kopetz and Günther Bauer. The time-triggered architecture. *Proceedings of the IEEE*, 91(1): 112–126, 2003.

[49] Kim Guldstrand Larsen and Bent Thomsen. A modal process logic. In *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS '88)*, pages 203–210. IEEE Computer Society, 1988. doi: 10.1109/LICS.1988.5119.

[50] Brandur Leach. Implementing Stripe-like idempotency keys in Postgres. `https://brandur.org/idempotency-keys`, October 2017. Published 2017-10-27; text checked against its source in github.com/brandur/sorg, accessed 2026-10-21.

[51] Jiapeng Li. Where does exactly-once live? Model, harness, and tool-contract effects on duplicate side effects in LLM agents, 2026.

[52] Marten Lohstroh, Christian Menard, Soroush Bateni, and Edward A. Lee. Toward a lingua franca for deterministic concurrent systems. *ACM Transactions on Embedded Computing Systems*, 20(4): 36:1–36:27, 2021. doi: 10.1145/3448128.

[53] Alan MacCormack, John Rusnak, and Carliss Y. Baldwin. Exploring the structure of complex software designs: An empirical study of open source and proprietary code. *Management Science*, 52(7): 1015–1030, 2006. doi: 10.1287/mnsc.1060.0552.

[54] Kiwan Maeng and Brandon Lucia. Supporting peripherals in intermittent systems with just-in-time checkpoints. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '19)*, pages 1101–1116. ACM, 2019. Pages from the NSF Public Access Repository record, https://par.nsf.gov/biblio/10129925.

[55] Robert C. Martin. OO design quality metrics: An analysis of dependencies. White paper, October 1994. Dated October 28, 1994.

[56] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Robert C. Martin Series. Prentice Hall, 2017. ISBN 978-0-13-449416-6.

[57] Thomas J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4): 308–320, 1976. doi: 10.1109/TSE.1976.233837.

[58] Eagon Meng and Daniel Jackson. What you see is what it does: A structural pattern for legible software. In *Proceedings of the 2025 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '25)*, pages 178–193. ACM, 2025. doi: 10.1145/3759429.3762628.

[59] Model Context Protocol authors. Model Context Protocol specification, revision 2025-03-26: Tools. `https://modelcontextprotocol.io/specification/2025-03-26/server/tools`, 2025. ToolAnnotations: readOnlyHint, destructiveHint, idempotentHint and openWorldHint, all declared hints; checked against schema/2025-03-26/schema.ts in github.com/modelcontextprotocol/modelcontextprotocol, accessed 2026-10-21.

[60] C. Mohan, Donald J. Haderle, Bruce G. Lindsay, Hamid Pirahesh, and Peter M. Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. doi: 10.1145/128765.128770.

[61] Suvam Mukherjee, Nitin John Raj, Krishnan Govindraj, Pantazis Deligiannis, Chandramouleswaran Ravichandran, Akash Lal, Aseem Rastogi, and Raja Krishnaswamy. Reliable state machines: A framework for programming reliable cloud services. In *33rd European Conference on Object-Oriented Programming (ECOOP 2019)*, volume 134 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:29. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2019. URL `https://drops.dagstuhl.de/opus/volltexte/2019/10810`.

[62] Narayana project. Narayana transaction manager, source code. `https://github.com/jbosstm/narayana`, 2026. Methods BasicAction.updateHeuristic (ArjunaCore) and SubordinateAtomicAction.doCommit (ArjunaJTA), main branch, accessed 2026-10-21.

[63] Edmund B. Nightingale, Peter M. Chen, and Jason Flinn. Speculative execution in a distributed file system. In *Proceedings of the 20th ACM Symposium on Operating Systems Principles (SOSP '05)*, pages 191–205. ACM, 2005. doi: 10.1145/1095810.1095829.

[64] Edmund B. Nightingale, Kaushik Veeraraghavan, Peter M. Chen, and Jason Flinn. Rethink the sync. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*, pages 1–14. USENIX Association, 2006.

[65] Rishiyur S. Nikhil. Bluespec system verilog: Efficient, correct RTL from high level specifications. In *Proceedings of the 2nd ACM/IEEE International Conference on Formal Methods and Models for Co-Design (MEMOCODE 2004)*, pages 69–70. IEEE, 2004. doi: 10.1109/MEMCOD.2004.1459818.

[66] Guy Pardon and Cesare Pautasso. Atomic distributed transactions: A RESTful design. In *Proceedings of the 23rd International Conference on World Wide Web (WWW '14 Companion)*, pages 943–948. ACM, 2014. doi: 10.1145/2567948.2579221.

[67] David Lorge Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.

[68] PCI-SIG. *PCI Express Base Specification, Revision 7.0*. PCI-SIG, 2025.

[69] Dewayne E. Perry and Alexander L. Wolf. Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4):40–52, 1992. doi: 10.1145/141874.141884.

[70] Protocol Buffers authors. descriptor.proto: the `idempotency_level` method option. `https://github.com/protocolbuffers/protobuf/blob/main/src/google/protobuf/descriptor.proto`, 2026. Values IDEMPOTENCY_UNKNOWN, NO_SIDE_EFFECTS and IDEMPOTENT; accessed 2026-10-21.

[71] Ganesan Ramalingam and Kapil Vaswani. Fault tolerance via idempotence. In *Proceedings of the 40th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '13)*, pages 249–262. ACM, 2013. doi: 10.1145/2429069.2429100.

[72] Restate. Restate documentation: Services, virtual objects and workflows. `https://docs.restate.dev/concepts/services`, 2026. Virtual objects: keyed, isolated K/V state, at most one handler with write access at a time per key, state committed atomically with the handler's execution; checked against docs/concepts/services.mdx in github.com/restatedev/documentation, accessed 2026-10-21.

[73] Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning, 2018. ISBN 978-1-61729-454-9.

[74] Jerome H. Saltzer, David P. Reed, and David D. Clark. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984. doi: 10.1145/357401.357402.

[75] Mary Shaw and David Garlan. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, Upper Saddle River, NJ, 1996. ISBN 0-13-182957-2.

[76] James E. Smith and Andrew R. Pleszkun. Implementation of precise interrupts in pipelined processors. In *Proceedings of the 12th Annual Symposium on Computer Architecture (ISCA '85), Boston, MA, USA, June 1985*, pages 36–44. IEEE Computer Society, 1985. doi: 10.1145/327010.327125.

[77] Smithy authors. Smithy 2.0 specification: Behavior traits. `https://smithy.io/2.0/spec/behavior-traits.html`, 2026. The readonly, idempotent and idempotencyToken traits; checked against docs/source-2.0/spec/behavior-traits.rst in github.com/smithy-lang/smithy, accessed 2026-10-21.

[78] Robert E. Strom and Shaula Yemini. Optimistic recovery in distributed systems. *ACM Transactions on Computer Systems*, 3(3):204–226, 1985. doi: 10.1145/3959.3962.

[79] Milijana Surbatovich, Limin Jia, and Brandon Lucia. I/O dependent idempotence bugs in intermittent systems. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA), 2019. doi: 10.1145/3360609.

[80] Temporal Technologies. Temporal events reference. `https://docs.temporal.io/references/events`, 2026. ActivityTaskScheduled, ActivityTaskStarted (recorded when the task is dispatched to a worker) and ActivityTaskTimedOut (a timeout failure that holds the timeout type); accessed 2026-10-21 via github.com/temporalio/documentation (events.mdx).

[81] Temporal Technologies. What is a Temporal retry policy? `https://docs.temporal.io/encyclopedia/retry-policies`, 2026. Activities retry by default, with unlimited maximum attempts; accessed 2026-10-21 via github.com/temporalio/documentation.

[82] Temporal Technologies. Detecting activity failures. `https://docs.temporal.io/encyclopedia/detecting-activity-failures`, 2026. Schedule-To-Start (until a worker picks up the task; never retried) and Start-To-Close timeouts; accessed 2026-10-21 via github.com/temporalio/documentation (detecting-activity-failures.mdx).

[83] Martin Thomson and Cory Benfield. HTTP/2. RFC 9113, June 2022. URL `https://www.rfc-editor.org/info/rfc9113`. Section 8.7 checked against the text at https://httpwg.org/specs/rfc9113.html.

[84] Vladislav Tomilov. Ballarch: Core 2.0 specification, reference implementations and benchmark. `https://github.com/4wl2d/Pokeball`, 2026. Version 2.0.0-rc.1. The repository keeps the project's earlier name, Pokeball, until it is renamed to Ballarch.

[85] Artem Trofimov and Boris Novikov. When tool calls succeed but workflows fail: Anomalies at the agent-tool boundary, 2026.

[86] Bas C. van Fraassen. Singular terms, truth-value gaps, and free logic. *The Journal of Philosophy*, 63 (17):481–495, 1966. doi: 10.2307/2024549.

[87] X/Open Company Ltd. *Distributed Transaction Processing: The XA Specification*. X/Open, 1991.

[88] Eren Yildiz, Saad Ahmed, Bashima Islam, Josiah Hester, and Kasim Sinan Yildirim. Efficient and safe I/O operations for intermittent systems. In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23)*, pages 63–78. ACM, 2023. doi: 10.1145/3552326.3587435.